

METAHEURÍSTICAS EVOLUTIVAS E ALGORITMO GENÉTICO

DCE797 - Algoritmos e Estruturas de Dados III

Atualizado em: 25 de maio de 2026

Iago Carvalho

Departamento de Ciência da Computação



METAHEURÍSTICAS COM MÚLTIPLAS SOLUÇÕES

Até agora, todas as heurísticas e metaheurísticas que vimos tinham uma única solução

- Esta solução sempre era reiniciada ou movida

Uma metaheurística evolutiva (ou evolucionária) otimiza um conjunto de soluções, chamado de população

- Uma população é composta por um conjunto de indivíduos
- Cada indivíduo é uma diferente solução

Cada indivíduo possui um conjunto de genes

- Um gene representa um elemento formador da solução

Estas múltiplas soluções, junto de mecanismos reprodutivos inspirados na teoria da seleção natural de Darwin, são os principais pontos para o desenvolvimento de algoritmos evolutivos

REPRESENTAÇÃO DE UMA SOLUÇÃO

No geral, uma solução contém n elementos

- Vértices
- Arestas
- Items
- ...

Deseja-se sempre ter uma representação de tamanho linear em relação ao número de elementos

REPRESENTAÇÃO DE UMA SOLUÇÃO



0	1	0	0	1	0	0
---	---	---	---	---	---	---



0	1	1	1	0	0	0
---	---	---	---	---	---	---

METAHEURÍSTICAS EVOLUTIVAS

A primeira metaheurística evolutiva foi o Algoritmo Genético

- Holland, John H. Adaptation in natural and artificial systems: an introductory analysis with applications to biology, control, and artificial intelligence. MIT press, 1992.

Desde então, metaheurísticas evolutivas tem se destacado como a *primeira tentativa* para se resolver um problema

- Quando você não sabe como resolver um problema, implementa uma metaheurística evolutiva e ela faz a sua *mágica* sozinha

Com o passar do tempo, uma infinidade de metaheurísticas evolutivas surgiram



<https://github.com/fcampelo/EC-Bestiarly>

Algoritmo 1: Algoritmo genético

Entrada: Conjunto de parâmetros // A ser definido

- 1 Inicialização da população
 - 2 Avaliação da população inicial
 - 3 **enquanto** *critério de parada não atingido* **faça**
 - 4 Seleção dos pais
 - 5 Cruzamento
 - 6 Mutação
 - 7 Avaliação das novas soluções geradas
 - 8 Geração da nova população
-

ALGORITMO GENÉTICO

ALGORITMO GENÉTICO - INICIALIZAÇÃO

O primeiro passo é gerar um conjunto de soluções iniciais P

- Utilização de heurísticas construtivas

Nesta etapa, é interessante que tenhamos soluções diversas

- Deste modo, heurísticas semi-aleatorizadas devem ser utilizadas

Outras estratégias também podem ser utilizadas

- Utilização de diversas heurísticas
- Geração completamente aleatória com reparo
- *Mix* de abordagens

Dizemos que cada indivíduo tem seu *fitness* (aptidão)

- Representa o quanto este indivíduo está apto a sobreviver no ambiente (problema)
- Pode ser de minimização ou maximização

A função de avaliação é responsável por computar o *fitness* de todos os indivíduos

- Para isto, deve-se considerar a função objetivo do problema
- Além disso, deve-se também considerar os corretos métodos de penalidades

ALGORITMO GENÉTICO - GERAÇÃO

Uma população se reproduz, gerando filhos. A este processo, damos o nome de **geração**

É o equivalente a uma iteração do algoritmo genético

Algoritmo 1: Algoritmo genético

Entrada: Conjunto de parâmetros // A ser definido

- 1 Inicialização da população
- 2 Avaliação da população inicial
- 3 **enquanto** *critério de parada não atingido* **faça**
- 4 Seleção dos pais
- 5 Cruzamento
- 6 Mutação
- 7 Avaliação das novas soluções geradas
- 8 Geração da nova população

Geração

Na seleção escolhemos pares de pais para o cruzamento

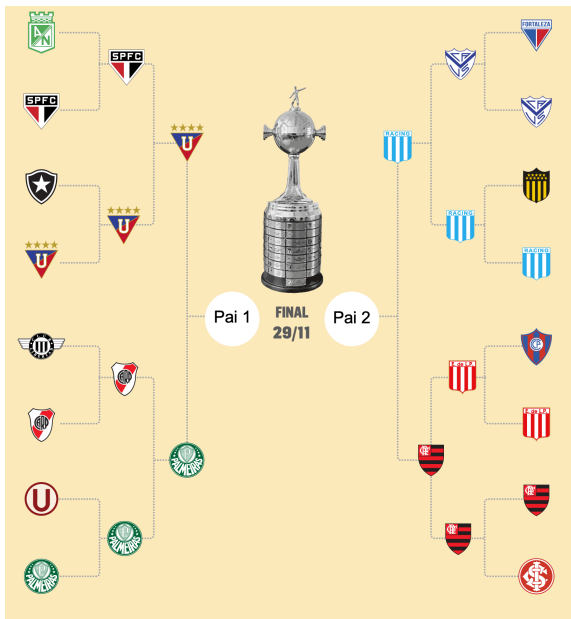
- Gerar filhos

Existem diversas maneiras de realizar a seleção de pais

Os mais comuns são

- Aleatório
- Torneio
- Roleta

ALGORITMO GENÉTICO - SELEÇÃO POR TORNEIO



ALGORITMO GENÉTICO - SELEÇÃO POR ROLETA

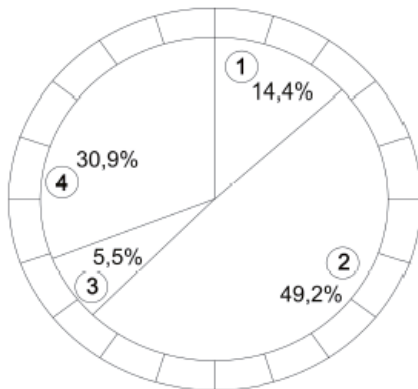


ALGORITMO GENÉTICO - SELEÇÃO POR ROLETA

Na verdade, a roleta é tendenciosa a escolha dos melhores indivíduos

- Se não for tendenciosa, seria uma seleção aleatória

0 1 1 0 1	→	169
1 1 0 0 0	→	576
0 1 0 0 0	→	64
1 0 0 1 1	→	361
4 indivíduos		Valor de fitness
Total = 1170		



ALGORITMO GENÉTICO - CRUZAMENTO

É uma forma de combinar dois indivíduos em um (ou outros dois) novos

- A ideia é trocar os elementos de duas soluções entre si
- Muitas vezes, gera soluções inválidas

O cruzamento é um método de intensificação

- Foca a busca em áreas intermediárias de duas soluções de boa qualidade

O método de cruzamento varia dependendo do problema

- Principalmente de sua codificação
 - Variáveis inteiras (e.g., caixeiro viajante)
 - Variáveis binárias (e.g., mochila binária)
 - Variáveis reais (e.g., funções matemáticas)

ALGORITMO GENÉTICO - CRUZAMENTO UNIFORME

Gera-se uma máscara binária de forma aleatória contendo uma posição para cada elemento da solução

- Se 1, faz a troca de elementos entre os pais
- Se 0, não faz nada

Máscara

0	0	1	0	0	1	1	1	0	0	1	0	1	1	0	1	0	0	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---



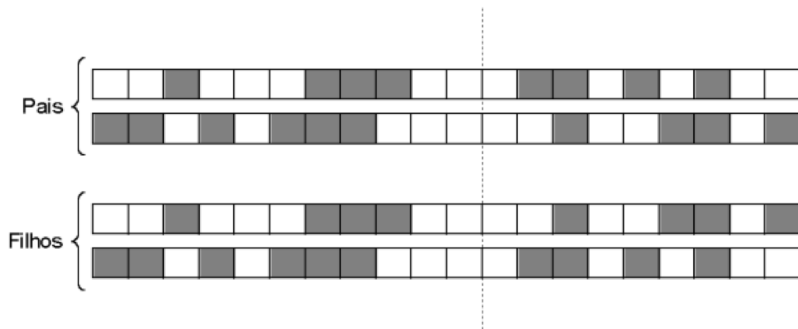
ALGORITMO GENÉTICO - CRUZAMENTO DE UM PONTO

A máscara binária é gerada com uma sequência de zeros e uma sequência de uns (ou o contrário)

- O ponto onde existe a troca de 0 para 1 é definido aleatoriamente

Máscara

0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---



ALGORITMO GENÉTICO - MUTAÇÃO

É uma troca aleatória em um ou mais genes da solução

- Número de genes definido aleatoriamente ou como parâmetro do algoritmo
- Funciona como mecanismo de diversificação

1	1	1	0	0	0
1	1	0	1	1	0

swap					
2	5	1	8	7	4
2	5	3	8	7	4

De forma alternativa, também pode-se utilizar um mecanismo de *shake* de busca local

ALGORITMO GENÉTICO - NOVA POPULAÇÃO

Ao definirmos uma nova população, damos fim a uma *geração*

Existem duas formas clássicas de gerar uma nova população

1. Evolução (menor pressão evolutiva)
2. Elitismo (maior pressão evolutiva)

Por **evolução**, descartamos todos os indivíduos da geração anterior e deixamos somente seus filhos

- Usualmente o número de indivíduos na população não é alterado

Por **elitismo**, fazemos um *ranking* de todas as soluções correntes

- População anterior e seus filhos
- Os $|P|$ melhores são selecionados
 - Os outros são descartados

ALGORITMO GENÉTICO - PARÂMETROS

Metaheurísticas evolucionárias possuem diversos parâmetros

- Difícil de realizar sua otimização
- Teremos uma aula sobre métodos de otimização de parâmetros

Em especial, um algoritmo genético possui como parâmetros

- Tamanho da população $|P|$
- Taxa de cruzamento
- Taxa de mutação
- Algoritmos de seleção
- Algoritmos de cruzamento
- Algoritmos de mutação
- Algoritmos de evolução
 - Evolução simples ou elitismo

Taxa de cruzamento: dado dois pais, qual é a probabilidade de aplicarmos o algoritmo de cruzamento neles?

- Caso não seja aplicado o cruzamento, dois filhos são gerados como seus *clones*

Taxa de mutação: qual a porcentagem dos genes serão mutados?

- Se muito pequeno, não faz efeito
- Se muito grande, equivale a uma busca aleatória

ALGORITMO GENÉTICO DE CHAVES ALEATÓRIAS

COMPLEXIDADE DO ALGORITMO GENÉTICO

Um algoritmo genético pode ser aplicado a, basicamente, qualquer problema de otimização

Apesar de simples e poderoso, sua implementação tem que corresponder exatamente ao problema que está a ser tratado

- Operadores de cruzamento e mutação específicos
- Operadores de reparo de solução

Desta forma, diversos esforços foram desenvolvidos para criar um algoritmo genético mais facilmente adaptável

ALGORITMO GENÉTICO DE CHAVES ALEATÓRIAS

O algoritmo genético de chaves aleatórias (RKGA, do inglês *Random Keys Genetic Algorithm*) foi criado com a proposta de ser um algoritmo genético padrão

Ele possui dois componentes

1. Componente independente do problema: Algoritmo genético
2. Componente dependente do problema: Decodificador

A principal vantagem do RKGA é de focar todo o desenvolvimento algorítmico no componente independente

- Para adaptar o algoritmo a outros problemas, basta algarer o componente dependente

COMPONENTE INDEPENDENTE DO PROBLEMA

O componente dependente é um algoritmo genético como vimos na aula passada

Algoritmo 2: Algoritmo genético

Entrada: Conjunto de parâmetros // A ser definido

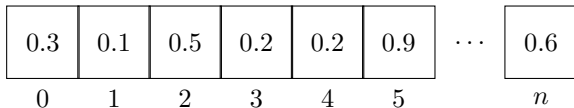
- 1 Inicialização da população
 - 2 Avaliação da população inicial
 - 3 **enquanto** *critério de parada não atingido* **faça**
 - 4 Seleção dos pais
 - 5 Cruzamento
 - 6 Mutação
 - 7 Avaliação das novas soluções geradas
 - 8 Geração da nova população
-

COMPONENTE DEPENDENTE DO PROBLEMA

O componente dependente é baseado em um vetor de chaves aleatórias e um decodificador

O vetor de chaves aleatórias, em geral, é um vetor de variáveis reais com n posições

- Cada posição representa um elemento da solução



Já o decodificador é um algoritmo que decodifica (ou transforma) o vetor de chaves aleatórias em uma solução válida

Esta operação de decodificação é dependente do problema

- Espera-se que seja de baixa complexidade computacional

Veremos alguns exemplos de decodificadores

DECODER PARA O CAIXEIRO VIAJANTE

Seja a um vetor de chaves aleatórias que representa uma solução para um problema do caixeiro viajante com 5 cidades

$$a = \langle \underset{A}{0.25}, \underset{B}{0.19}, \underset{C}{0.67}, \underset{D}{0.05}, \underset{E}{0.89} \rangle$$

Como podemos transformar este vetor em uma solução para o problema do caixeiro viajante?

DECODER PARA O CAIXEIRO VIAJANTE

Seja a um vetor de chaves aleatórias que representa uma solução para um problema do caixeiro viajante com 5 cidades

$$a = \langle \underset{A}{0.25}, \underset{B}{0.19}, \underset{C}{0.67}, \underset{D}{0.05}, \underset{E}{0.89} \rangle$$

Como podemos transformar este vetor em uma solução para o problema do caixeiro viajante?

Basta ordenar as chaves (complexidade $\mathcal{O}(n \log n)$)

$$a = \langle \underset{D}{0.05}, \underset{B}{0.19}, \underset{A}{0.25}, \underset{C}{0.67}, \underset{E}{0.89} \rangle$$

Isto nos gera a rota $D \rightarrow B \rightarrow A \rightarrow C \rightarrow E \rightarrow D$

DECODER PARA O PROBLEMA DA MOCHILA BINÁRIA

Considere a mesma chave aleatória a

$$a = \langle \underset{A}{0.25}, \underset{B}{0.19}, \underset{C}{0.67}, \underset{D}{0.05}, \underset{E}{0.89} \rangle$$

Como podemos transformar este vetor em uma solução para o problema da mochila binária?

Existem duas maneiras simples

- Uma envolve ordenação
- A outra não

DECODER PARA O PROBLEMA DA MOCHILA BINÁRIA

Primeiro método: ordenação

$$a = \langle \underset{D}{0.05}, \underset{B}{0.19}, \underset{A}{0.25}, \underset{C}{0.67}, \underset{E}{0.89} \rangle$$

Podemos fazer um *loop* neste vetor ordenado inserindo (em ordem) todos os itens que cabem na mochila

Segundo método: corte

Todos os itens com valor de chave maior que δ são selecionados

Para $\delta = 0.5$, selecionamos os itens $\{C, E\}$

Este segundo método pode gerar soluções inválidas

- Um mecanismo de reparo é necessário

DECODER PARA O PROBLEMA DA SOMA DE SUBCONJUNTOS

O problema da soma de subconjuntos é um problema de decisão

Dado um conjunto de valores S , devemos selecionar $S' \subset S$ tal que

$$\sum_{s \in S'} x_s = T$$

Considere $T = 10$ e o vetor de chaves aleatórias a como abaixo.

Além disso, seja $x_a = 1, x_b = 7, x_c = 8, x_d = 4, x_e = 9$. Como você resolveria o problema?

$$a = \langle \underset{A}{0.25}, \underset{B}{0.19}, \underset{C}{0.67}, \underset{D}{0.05}, \underset{E}{0.89} \rangle$$

ALGORITMO GENÉTICO DE CHAVES ALEATÓRIAS TENDENCIOSAS

O RKGA é um algoritmo interessante, mas não é utilizado na prática

Entretanto, seus conceitos são amplamente utilizados pelo Algoritmo Genético de Chaves Aleatórias Tendenciosas

- BRKGA, do inglês *Biased Random Keys Genetic Algorithm*

O BRKGA utiliza os mesmos conceitos de chaves aleatórias e decodificação

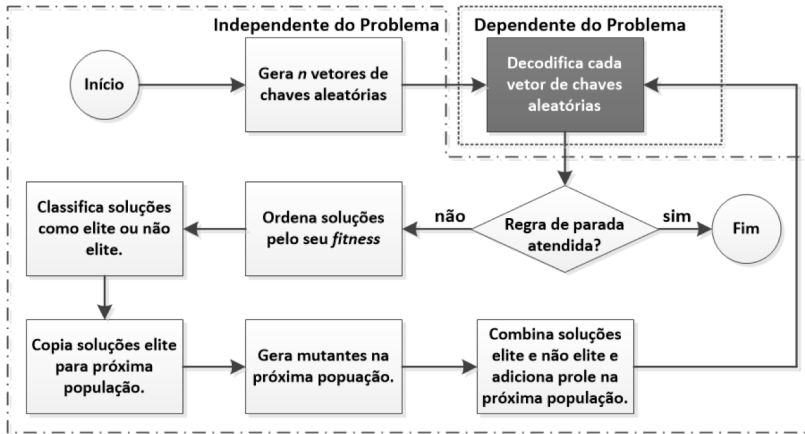
Além disso, ele fornece um *framework* para implementação do algoritmo genético quando associado a estas chaves aleatórias

- Operadores de mutação e cruzamento especializados

FLUXOGRAMA DO BRKGA

O BRKGA pode ser sumarizado pelo seguinte fluxograma

- Soluções elite são aquelas com melhor qualidade de função objetivo



CRUZAMENTO DO BRKGA



CRUZAMENTO DO BRKGA

O cruzamento seleciona dois pais, sendo um da população elite e outro do restante da população

O BRKGA costuma utilizar um cruzamento uniforme como o Algoritmo Genético tradicional

Máscara

0	0	1	0	0	1	1	1	0	0	1	0	1	1	0	1	0	0	1	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---



PARÂMETROS DO BRKGA

Assim como o Algoritmo Genético, a grande dificuldade do BRKGA é trabalhar sua parametrização

Tamanho da população (P): Recomenda-se $P = aN$, onde $a \in \mathbb{R}_{\geq 1}$ e N é o tamanho do cromossomo

Tamanho da população elite (P_e): Recomenda-se que $0,10P \leq P_e \leq 0,25P$

Tamanho da população mutante (P_m): Recomenda-se que $0,1P \leq P_m \leq 0,3P$

Probabilidade de herança do elite (c_e): Recomenda-se que $0,5 < c_e \leq 0,8$